

lower_bound / upper_bound 정리

정렬된 범위에서 기준값의 위치를 이분 탐색으로 찾는 알고리즘입니다.

사용처 (Where)

- 정렬된 데이터에서 빠르게 위치 찾기
- 특정 값 이상/초과 첫 위치 찾기
- 값의 개수 계산

방법 (How to Use)

- 반드시 정렬된 범위에 사용
- lower_bound는 x 이상 첫 위치
- upper_bound는 x 초과 첫 위치
- 반환 iterator와 begin의 거리로 인덱스 계산

기본 정보

- 필요 헤더: `#include <algorithm>`
- 기본 선언: `auto it = lower_bound(v.begin(), v.end(), x);`

왜 써야 할까? (Why)

- 정렬된 데이터 검색을 빠르게 처리
- 개수/삽입 위치 계산에 유용
- 직접 이분 탐색 구현보다 실수 가능성이 낮음

주요 함수

함수/문법	의미
lower_bound	x 이상 첫 위치
upper_bound	x 초과 첫 위치
equal_range	둘을 한 번에
distance	위치 계산

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 2, 4, 5};
    auto it = lower_bound(v.begin(), v.end(), 2);

    cout << (it - v.begin());
}
```

lower_bound / upper_bound 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
vector lower/upper	$O(\log n)$
list에서 사용	이동 비용 때문에 비효율
정렬 비용	$O(n \log n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	lower_bound / upper_bound 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 특정 값의 개수 구하기

upper_bound 위치에서 lower_bound 위치를 빼면 정렬된 배열에서 해당 값의 개수를 얻습니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 2, 2, 4};
    int x = 2;

    auto lo = lower_bound(v.begin(), v.end(), x);
    auto hi = upper_bound(v.begin(), v.end(), x);

    cout << (hi - lo);
}
```

처음 배울 때 자주 하는 실수

- 정렬하지 않은 데이터에 쓰면 결과가 의미 없음
- 못 찾았다는 뜻이 아니라 삽입 위치를 반환
- end 반환 가능성을 확인

비슷한 항목과 차이

- `binary_search`는 존재 여부만 `bool`로 반환
- `find`는 정렬 없이 선형 검색
- `set/map`의 `lower_bound`는 멤버 함수 사용 가능

한 줄 정리: `lower_bound / upper_bound`은(는) '정렬된 데이터에서 빠르게 위치 찾기' 상황에서 먼저 떠올리면 좋은 STL입니다.